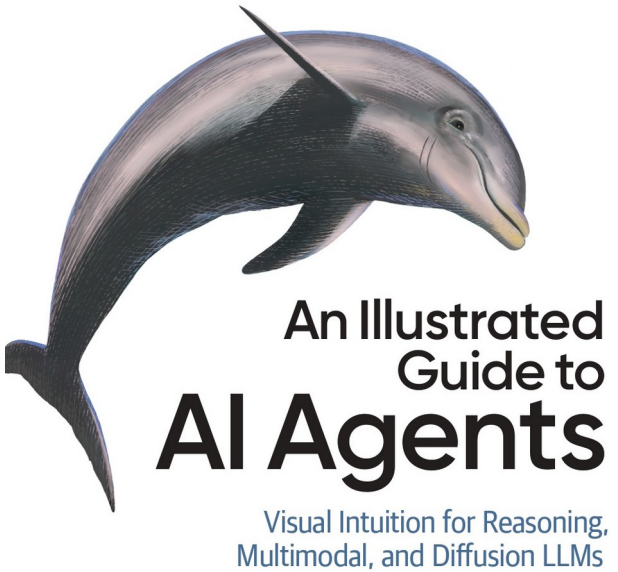
M4

Tools & MCP

Module 4 — Letting the agent act on the world



307307 — BI Methods for Generative AI

Agentic AI · University of Petra

Lecturer: _____ Date: _____

Learning Objectives

1. Explain what a tool is and why an LLM needs one to act.
2. Trace the tool-calling flow: create, define, select, call, process output.
3. Explain intent vs action — and why running the tool ourselves is a safety boundary.
4. Describe how a tool is defined to the LLM (prompted vs native schemas).
5. Explain what MCP is and the problem it solves; recognise Skills.

Where We Are — Letting It Act

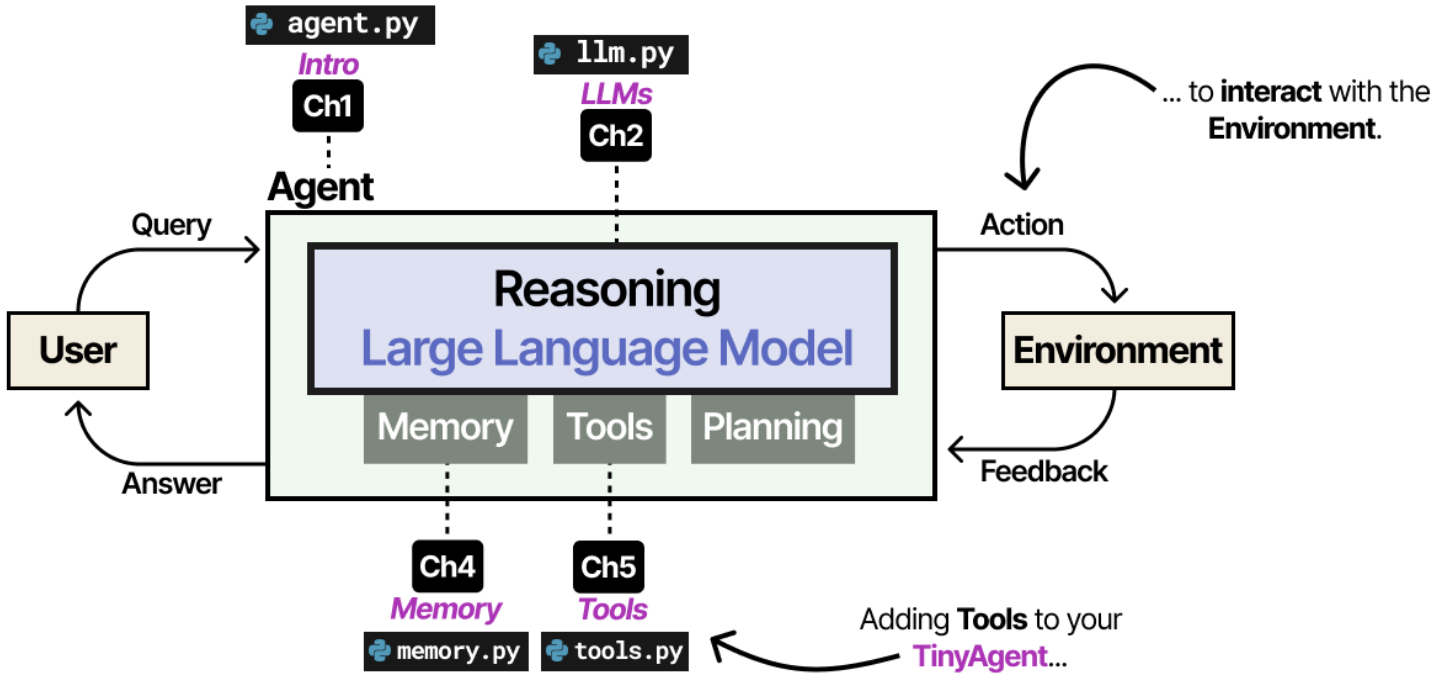


Figure from An Illustrated Guide to AI Agents (O'Reilly, 2026)

Can think (M2)

a reasoning brain

Can remember (M3)

holds a conversation

Can act (today)

uses tools to affect the world

What Is a Tool — and Why?

A plain LLM is text-in / text-out: it can describe wanting to act, but it cannot act. A tool is a function or API we let the agent use.

Take an action

Change something: send an email, write a file, schedule an appointment, run code.

Retrieve information

Fetch something: web search, read a document, query a database or an API.

Without tools, an agent can only think and plan — not act.

The Tool-Calling Flow

1. **Create** — write the function (document it well)
2. **Define** — tell the LLM the tool exists and how to call it
3. **Select** — the LLM decides which tool fits (if any)
4. **Call** — the LLM outputs a structured tool call
5. **Process** — we run the tool and feed the result back

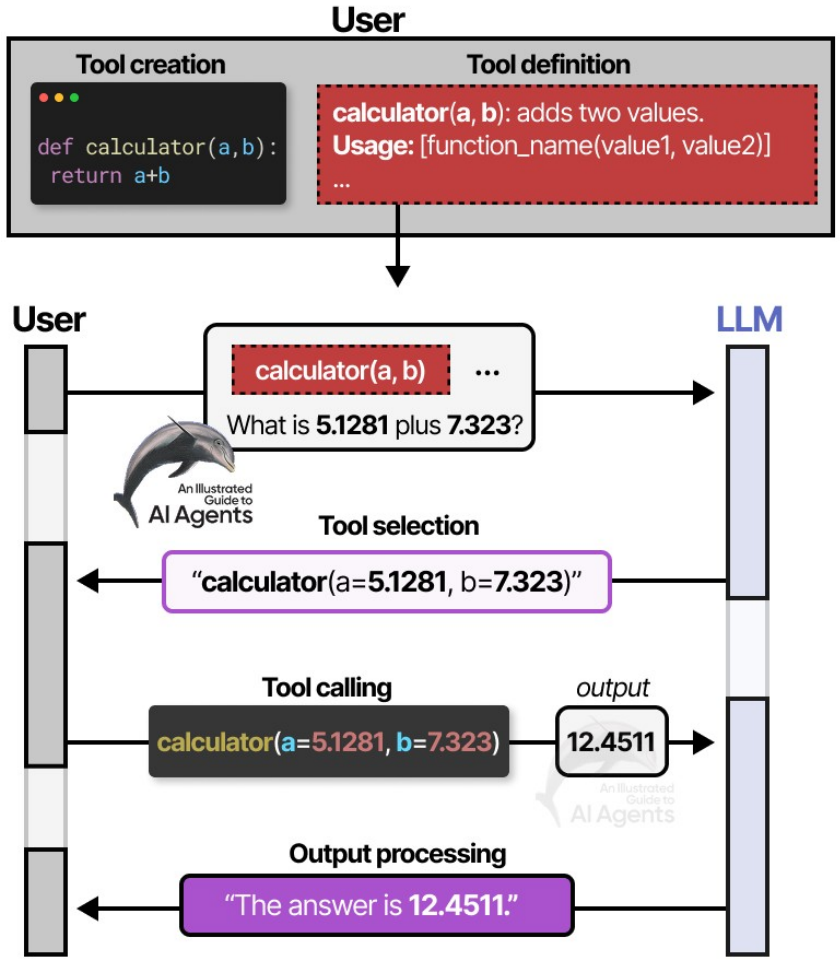


Figure 5-5 (O'Reilly, 2026)

Intent vs Action — the Safety Boundary

The key idea: when the LLM “calls” a tool, it only outputs text expressing the intent. Nothing happens until our code runs it.

```
{"tool": "multiply",
 "kwargs": {"a": "5.1", "b": "7.3"}}
```

The LLM outputs this text — it is a request, not an action.

A request is not permission.

Our code decides whether to run it — and can require human approval for risky tools (delete a file, send a payment) before anything happens.

Analogy

The LLM is like an assistant who writes “please wire $500 to this account” on a note. Nothing moves until a person — your code — actually does it, and your code can refuse.

How the LLM Is Told About Tools

Prompting (in-context)

- List the tools in the system prompt and state the reply format.
- A small parser reads the reply and extracts the call.
- **Simple and transparent — what we use in the lab.**

Native / schemas

- Newer models accept a formal JSON schema of each tool.
- They emit tool calls directly.
- **Cleaner and more reliable — but more of a black box.**

Good practice: document tools well · keep fewer than ~10 · keep each tool's scope narrow.

Think, Act, Observe

After a tool runs, we add its result back to the conversation as an OBSERVATION and let the LLM continue. Looping over these steps is the ReAct pattern — the next module.

1

THOUGHT

what the model decides to do

2

ACTION

the tool call it emits

3

OBSERVATION

the tool's result, fed back

4

ANSWER

the final reply

Today the agent uses a tool once. Next module: it repeats the loop to solve multi-step tasks.

MCP — a Standard Plug for Tools

The N × M problem: every model talks to tools differently, so each model-tool pair needs its own integration.

MCP (open standard, Anthropic, 2024) fixes this — the "USB-C of AI." One standard connection, and the tool provider maintains it. N + M, not N × M.

- **Host** — the app using the LLM (our agent)
- **Client** — keeps a 1-to-1 link to a server
- **Server** — exposes a tool / data source

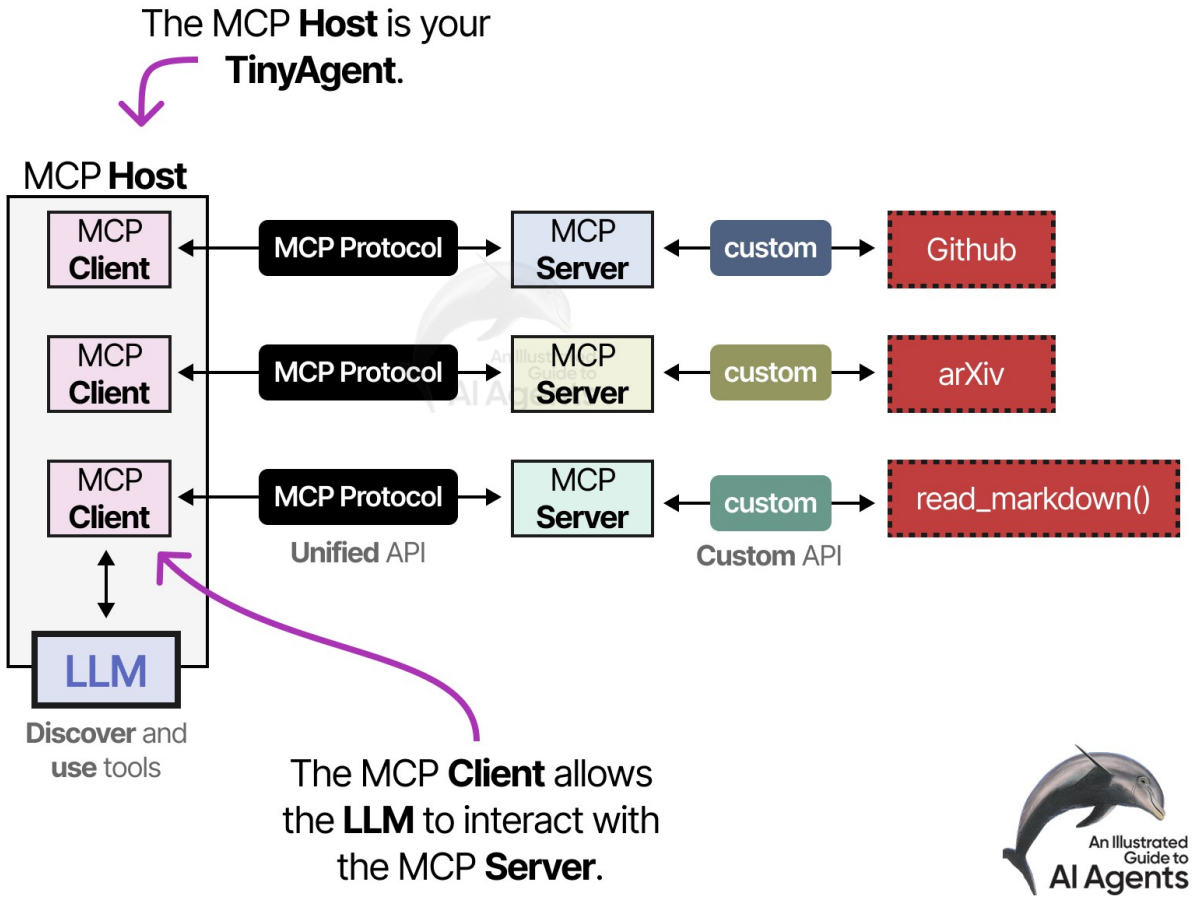


Figure 5-29, An Illustrated Guide to AI Agents (O'Reilly, 2026)

Skills — Recipes for Recurring Jobs

Tools tell an agent what it can do. Skills tell it how to get a recurring job done — a recipe of steps, conventions, and references for your context.

Progressive disclosure — load only what's needed, to keep the context lean:

1	**name + description**	always loaded (~100 tokens)
2	**the full instructions**	loaded when the skill is activated
3	**extra files / scripts**	loaded only when needed

This is the mechanism behind the skills you may already use with Claude.

What You Will Build Today

1 Create tools

two Python functions: a calculator and a weather lookup.

2 Define them

describe the tools in the system prompt + a JSON reply format.

3 Parse & execute

read the LLM's tool call, run the function ourselves.

4 Observe & answer

feed the result back as an OBSERVATION; the agent replies.

A run-and-observe lab: watch the intent, the action, then the answer. One small exercise at the end.

Key Takeaways

1. A tool lets the agent act; without tools it can only think and plan.
2. Tool use has five steps: create, define, select, call, process output.
3. The LLM emits only the intent — our code executes it. That gap is the safety boundary.
4. Tools are defined by prompting or by native JSON schemas.
5. MCP standardises tool connections (the USB-C of AI); Skills package how to do recurring jobs.

Discussion

Q1 Why can't a plain LLM use a tool on its own? What does it actually produce?

Q2 Explain 'intent vs action' and why it is a safety boundary. Where would human approval fit?

Q3 What problem does MCP solve, and what are its three components?

Next: Planning & ReAct — the agent loops through think, act, observe to solve multi-step tasks.